

Uncertainty and probability

- $P(A|B) = \frac{P(A \cap B)}{P(B)}$ — conditional probability
(A probability of an event a, given that another event (B) has already occurred)
- $P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$ — Bayes' Theorem

• classification: classification is a form of data analysis that extracts models describing data classes.

- A classifier/classification model
 - ↳ predicts categorical labels (classes)
- Two steps → (i) learning (ii) classification

• Decision tree induction:

- Top down recursive tree induction algorithm

- uses attribute selection measure

- ↳ To select the attribute tested for each non-leaf node in the tree.

- Internal node (non-leaf node)

- ↳ Denotes a test on an attribute

- Branch

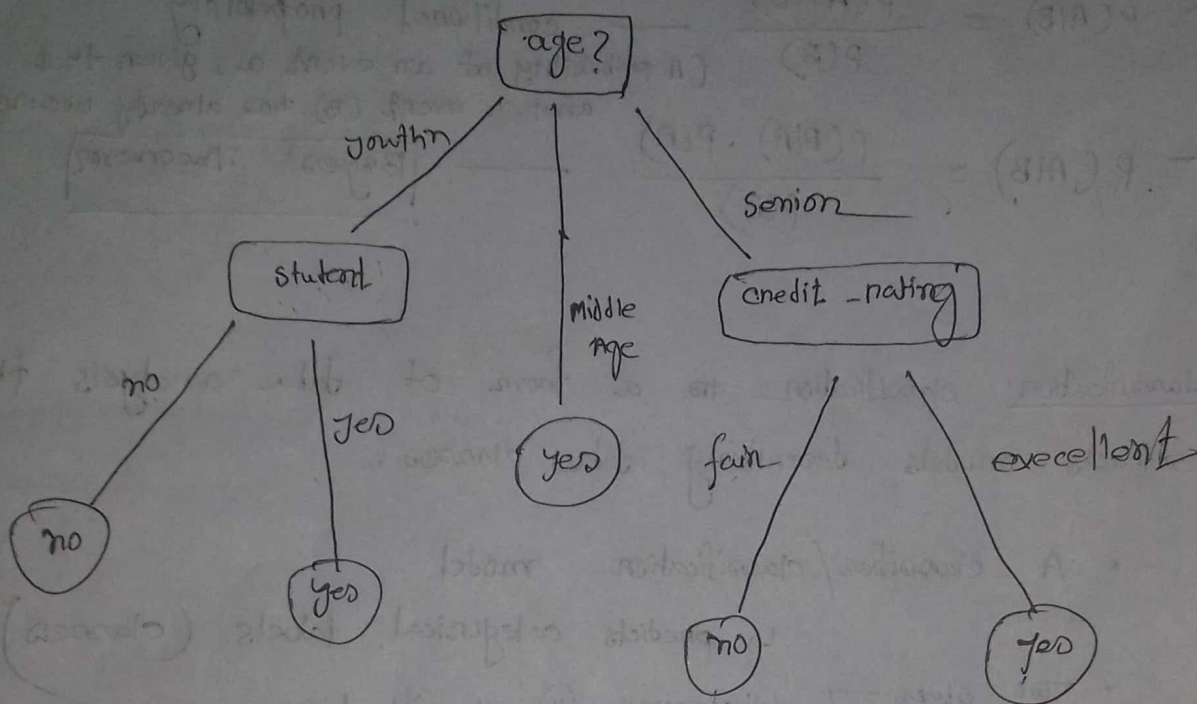
- ↳ outcome of the test

- leaf node / terminal node

- ↳ holds a class label

Example:

A Decision tree for the concept buys-computer.
senior, excellent



Bayesian classifier

- Naive Bayesian classifier assume that, the effect of an attribute value on a given class is independent of the values of the other attributes. This assumption is called class-conditional Independence.

- It is made to simplify the computations involved and in this sense, is considered "Naive".

□ Prior probability

- ↳ Probability of hypothesis before observing the evidence.
- ↳ This is the best rational assessment of the probability of an outcome based on the current knowledge before an experiment is performed.

$$P(A) = \frac{\text{Number of Desired events (A)}}{\text{Total number of events occurred}}$$

□ posterior probability

- ↳ is a type of conditional probability that results from updating the prior probability with information summarized by the likelihood, through the application of Bayes theorem.

posterior probability $\leftarrow$
$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$
 (prior probability)

□ predicting a class level using naive bayesian classification.

$X = (\text{age} = \text{youth}, \text{income} = \text{medium}, \text{student} = \text{yes}, \text{credit-rating} = \text{fair})$

$$P(\text{buys-computer} = \text{yes}) = 9/14 = 0.643$$

$$P(\text{buys-computer} = \text{no}) = \frac{5}{14} = 0.357$$

Now we complete the following probabilities:

$$P(\text{age} = \text{youth} \mid \text{buys} - \text{computer} = \text{yes}) = \frac{2}{9} = 0.222$$

$$P(\text{age} = \text{youth} \mid \text{buys} - \text{computer} = \text{no}) = \frac{3}{5} = 0.600$$

$$P(\text{income} = \text{medium} \mid \text{buys} - \text{computer} = \text{yes}) = \frac{4}{9} = 0.444$$

$$P(\text{income} = \text{medium} \mid \text{buys} - \text{computer} = \text{no}) = \frac{2}{5} = 0.400$$

$$P(\text{student} = \text{yes} \mid \text{buys} - \text{computer} = \text{yes}) = \frac{6}{9} = 0.667$$

$$P(\text{student} = \text{yes} \mid \text{buys} - \text{computer} = \text{no}) = \frac{1}{5} = 0.200$$

$$P(\text{credit} - \text{rating} = \text{Fair} \mid \text{buys} - \text{computer} = \text{yes}) = \frac{4}{9} = 0.667$$

$$P(\text{credit} - \text{rating} = \text{Fair} \mid \text{buys} - \text{computer} = \text{no}) = \frac{2}{5} = 0.400$$

using these probabilities, we obtain:

$$\begin{aligned} P(X \mid \text{buys} - \text{computer} = \text{yes}) &= 0.222 \times 0.444 \times 0.667 \times 0.667 \\ &= 0.444 \end{aligned}$$

$$\begin{aligned} P(X \mid \text{buys} - \text{computer} = \text{no}) &= 0.600 \times 0.400 \times 0.200 \times 0.400 \\ &= 0.016 \end{aligned}$$

now we compute:

$$p(x | bu-c = \text{yes}) \cdot p(bu-c = \text{yes}) = 0.444 \times 0.643 = 0.028$$

$$p(x | bu-c = \text{no}) \cdot p(bu-c = \text{no}) = 0.010 \times 0.357 = 0.007$$

□ How to construct a Naive Bayes' classifier

1. Let D be a training set of tuples and their associated class labels.

• Each tuple is represented by an n -dimensional vector,

$$X = (x_1, x_2, \dots, x_n).$$

2. Naive Bayesian classifier predicts that tuple X belongs to the class c_i if and only if,

$$p(c_i | X) > p(c_j | X) \text{ for } 1 \leq j \leq n, j \neq i$$

Thus, we maximize $p(c_i | X)$. The class c_i for which $p(c_i | X)$

~~$p(c_i | X)$~~ is maximized is called maximum posteriori hypothesis.

By Bayes' theorem:

$$p(c_i | X) = \frac{p(X | c_i) p(c_i)}{p(X)}$$

To summarize we need to follow the below steps $\rightarrow$

- Convert the given dataset into frequency tables.
- Generate Likelihood table by finding the probabilities of given features.
- ~~at~~ at last, use Bayes theorem to calculate the posterior probability.

Chain Rule:

conditional probability for $P(A|B)$ results in product Rule

$$P(A|B) = P(A|B)P(B)$$

by ~~repeating~~ repeated application of the above rule

for n variables we obtain the chain rule —

$$P(X_1, \dots, X_n)$$

$$= P(X_n | X_1, \dots, X_{n-1}) P(X_1, \dots, X_{n-1})$$

$$= P(X_n | X_1, \dots, X_{n-1}) P(X_{n-1} | X_1, \dots, X_{n-2}) \dots P(X_1)$$

$$= \prod_{i=1}^n P(X_i | X_1, \dots, X_{i-1})$$

marginization:

$$P(A) = P((A \cap B) \cup (A \cap \bar{B}))$$

$$P(X_1 = x_1, \dots, X_{d-1} = x_{d-1})$$

$$= \sum_{x_d} P(X_1 = x_1, \dots, X_{d-1} = x_{d-1}, X_d = x_d)$$

The application of this formula is called marginalization

Bayes's Theorem:

According to conditional theorem, we can say

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad P(B|A) = \frac{P(A \cap B)}{P(A)}$$

by solving both equations for $P(A \cap B)$ and equating them we obtain Bayes's theorem.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Q

	Toothache		¬Toothache	
	catch	¬catch	catch	¬catch
cavity	0.108	0.012	0.072	0.008
¬cavity	0.016	0.064	0.144	0.576

$$P(\text{cavity} | \text{toothache}) = ?$$

$$P(\text{cavity} | \text{toothache} \vee \text{catch}) = ?$$

$$\begin{aligned} P(\text{cavity}) &= 0.108 + 0.012 + 0.072 + 0.008 \\ &= 0.2 \end{aligned}$$

$$\begin{aligned} P(\neg \text{cavity}) &= 0.016 + 0.064 + 0.144 + 0.576 \\ &= 0.8 \end{aligned}$$

$$\begin{aligned} P(\text{Toothache}) &= 0.108 + 0.012 + 0.016 + 0.064 \\ &= 0.2 \end{aligned}$$

$$\begin{aligned} P(\neg \text{Toothache}) &= 0.072 + 0.008 + 0.144 + 0.576 \\ &= 0.8 \end{aligned}$$

$$P(\text{cavity} \cup \text{toothache}) = P(C) + P(T) - P(C \cap T)$$

$$= 0.2 + 0.2 - (0.108 + 0.012)$$

$$= 0.28$$

$$P(\text{cavity} | \text{toothache}) = \frac{P(C \cap T)}{P(T)}$$

$$= \frac{0.108 + 0.012}{0.2}$$

$$= 0.6$$

$$P(\text{cavity} | \text{toothache} \vee \text{catch}) = \frac{P(C) + P(T)}{P(T \vee C)}$$

$$= \frac{P(\text{cavity} \cap (\text{tooth} \vee \text{catch}))}{P(T \vee C)}$$

$$= \frac{0.108 + 0.012 + 0.072}{0.28} = 0.685$$

Previous Year

• Decision Theory: Decision theory is a logical study of how decisions are made in a structure or system where decision environment is uncertain and decision variables are unknown.

Random variable:

- A Random variable is defined as a measurable function from a probability measure space (sample space) to a measurable space.

conditional probability:

marginal probability:

(b)

probability of liver disease = $10\% = .10 = P(L)$

" " being alcoholic = $5\% = .05 = P(A)$

probability of ^{have} liver disease ^{and} being ^{alcoholic} = $7\% = .07 = P(A \cap L)$

we must calculate,

$$P(L|A) = \frac{P(A \cap L)}{P(A)} \times \frac{P(L)}{P(A)} = \frac{0.07 \times 0.10}{0.05} = 0.14$$

$$\frac{0.07 \times 0.10}{0.05} = 0.14$$

14%

Chap-6

constraint satisfaction problem

7] constraint satisfaction: Sudoku Solver.

- variable n : Each one the 81 squares
- Domains: Each number 1 to 9 (i.e. - 1, 2, 3, 4, 5, 6, 7, 8, 9)
- constraints:
 1. Fill a 9×9 grid with digits 1 to 9.
 2. Each column contains all digits from 1 to 9
 3. Each row contains all " " 1 to 9
 4. Each 3×3 box contains all " " 1 to 9.

To illustrate CPS in Sussku let's showcase an example.

2	3	5					9
9	2						
	7	8					
1	9						3

- Our aim is to fill the shaded box with suitable digit within the domain's 1 to 9

- Now by applying AC-3 algorithm, A_1 domain cannot have values 2, 3, 5, 7, 8, 9 because it ~~has~~ those are already ~~app~~ present in the 3×3 box.

- It cannot have value 1 because it is already present in the 1st column.

so the possible digits are 4 and 6 for A_1 .

The AC-3 Algorithm:

function AC-3(csp) returns false if an inconsistency is found and true otherwise.

Inputs: csp, a binary csp with components (X, D, c)

local variables: queue, a queue of arcs, initially all the arcs in csp.

while queue is not empty do

$(X_i, X_j) \leftarrow \text{Remove-First}(\text{queue})$

if REVISE(csp, X_i, X_j) then

if size of $D_i = 0$ then return false

for each X_k in X_i 's NEIGHBORS - $\{X_j\}$ do

add (X_k, X_i) to queue

return true

function REVISE(csp, X_i, X_j) returns true iff we revise the domain of X_i

revised $\leftarrow$ false

for each x in D_i do

if no value y in D_j allows (x, y) to satisfy the

constraint between X_i and X_j . then

delete x from D_i

revised $\leftarrow$ true

return revised.

chapter-6

• Constraint: (Restriction, condition)

↳ Have alternative ways but bound to do the task in a certain way.

• CSP: A problem is solved when each variable has a value that satisfies all the constraints on the variable.

A problem described this way is called a constraint satisfaction problem.

CSP is used to solve a wide variety of problems efficiently.

□ problem Formulation:

① A constraint satisfaction problem consists of three components:
 $\therefore X, D, C$.

X is a set of variables, $\{x_1, \dots, x_n\}$

D is a set of domains $\{D_1, \dots, D_n\}$

C is a set of constraints.

• A problem is solved when each variable has a value that satisfies all the constraints on that variable is called

CSP.

→ CSP de

• can

□ state

1. constraint

↳ wh

2. comp

↳

3. partial

↳

□ map

□ Scope:

□ Relation:

CSP.

→ CSP deals with general problem instead of specific problems,
• can solve complex problem.

State in CSP: state of CSP defined by assignment.

1. consistent and legal assignment:

↳ when an assignment doesn't violate any constraint.

2. complete assignment:

↳ A complete assignment is one in which every variable is assigned.

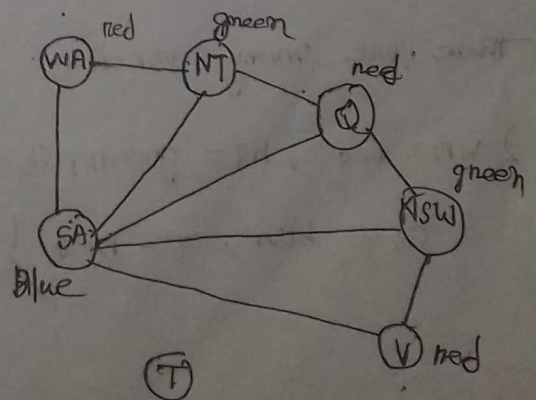
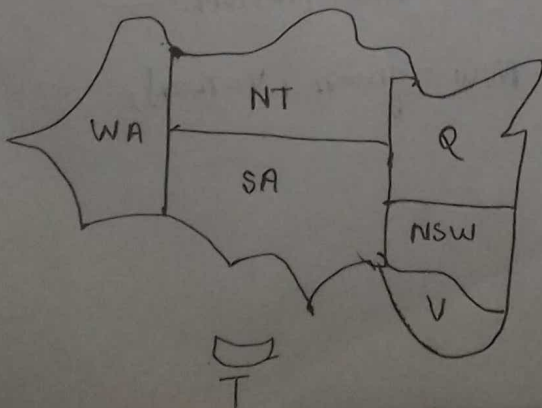
3. partial assignment:

↳ partial assignment is one that assigns values to only some of the variables.

Map Coloring:

Scope: A tuple of variables that participate in the constraints.

Relation: A relation that defines the values that those variables can take on.



We are looking at a map of Australia showing each of its states and territories. We have to color each region either red, green or blue, in such a way that no neighbouring regions have the same color.

To formulate this as a CSP, we define the variables to be the region:

$$X = \{WA, NT, Q, NSW, V, SA, T\}$$

The domain of each variable $D_i = \{red, green, blue\}$

As there are nine places where region borders, there are nine constraints.

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, \\ WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}$$

$SA \neq WA$ is a shorthand for $\{(SA, WA), SA \neq WA\}$

$$\{(red, green), (red, blue), (green, red), (green, blue), \\ (blue, red), (blue, green)\}$$

There are many possible solutions to this problem!

$$\{WA = red, NT = green, Q = red, NSW = green, V = red, \\ SA = blue, T = red\}$$

chapter-4

▣ Local Search: Local search in AI is an optimizing algorithm to find the optimal solution more quickly. Local search algorithms are used when we care only about a solution but not the path to a solution.

▣ Why do we use local search strategy to address optimization problem?

Q show how the last configuration of 4-queens on a 4×4 board has fewer conflicts than the first configuration using local search strategy in where conflicts means there are no two queens on the same row, column or diagonal.

conflict: conflict means there are no two queens on the same row, column or diagonal.

$\Rightarrow$ 4 queens problem is to place 4 queen on a 4×4 chessboard in such a manner that no queen attack each other being in the same row, column or diagonal.

$\Rightarrow$ Here we have to place 4 queen Q_1, Q_2, Q_3, Q_4 on the 4×4 chessboard such that no queens attack each other.

It can be solved using local search approach —

Approach: 1. put Q_1 at position (1,1). Now we can't put Q_2 in row 1, because they will conflict.

2. we can put Q_2 in row 2, in column 3 at (2,3). If we do that there will be no option for placing Q_3 in row 3.

⊗ so we backtrack and put Q_2 at $(2,4)$.

As a result we will be able to put Q_3

at row: 3 at $(3,2)$

⊗ But if we do that there will be no option
for placing Q_4 .

3. so we have to back track and place Q_2 at

$(3,2)$, $Q_2(2,4)$, $Q_3(3,1)$ and $Q_4(4,3)$.

so after final approach:

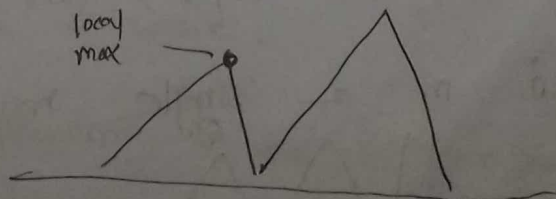
	Q_1		
			Q_2
Q_3			
		Q_4	

Q2) what are the problems of hill-climbing algorithm for getting stuck? How to escape the problems using the simulated - annealing search?

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing ~~the~~ elevation / value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbor has a higher value.

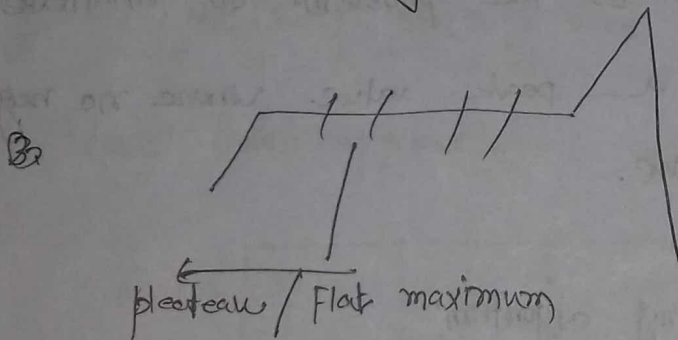
— Problems in hill climbing algorithm:

1. Local maximum: A local maximum is a peak state in the landscape which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.



2. plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contains the same value, because of the this algorithm does not find any best direction to move.

A hill climbing search might get lost.



3. Ridged: A ridge is a special form of the local maximum. It has area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.



→ How to escape?

- A simulated annealing algorithm is a method to solve bound-constrained and unconstrained optimization parameter models.
- In every simulated annealing example, a random new point is generated. The distance between the current point and the new point has a basis of the probability distribution on the scale of the proportion of temperature.
- There is a huge difference between hill-climbing and simulated-annealing considering the way they are applied, and the results are achieved.

where hill-climbing achieves optimum value by tracking the current state of neighbourhood.

S-A achieves the objective by selecting the best move once a while.

A global optimum solution is guaranteed with simulated annealing.

As simulated annealing accepts the worse solution once in a while to jump out of the local optimum,

It solves the problem of ~~the~~ hill climbing being stuck in local optimum point.

chapter-3

Q1] Mention the advantages of Informed Search than uninformed Search.

Basis of comparison	Informed Search	Uninformed Search
Basic Knowledge	• Uses knowledge to find the steps to find solution	• Do not use knowledge
Efficiency	• Highly efficient as consumes less time and cost.	• Efficiency is mediatory
Cost	Low	Comparatively high
Performance	• Finds the solution more quickly	• Speed is slower.
Algorithm	• A*, heuristic BFS, DFS	• BFS, DFS

Q2] What is heuristic function of informed search strategy?

- Heuristic is a function which is used in informed search and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.

The heuristic method, however, might not always give the best solution, but guaranteed to find a good solution in

in reasonable time.

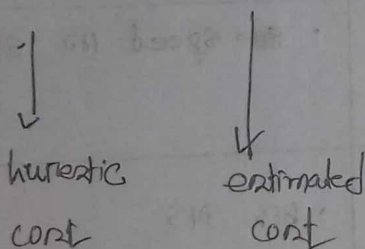
→ It is represented by $h(n)$

- $h(n)$ is always positive

- It calculates the cost of an optimal path between the pair of states.

• Admissibility of the heuristic function —

$$h(n) \leq h^*(n)$$



□ How to minimize the total estimated solution cost using the best first search, A^* search algorithm

⇒ minimizing the total estimated solution cost:

The most widely known form of best first search is called A^* search.

A^* search evaluates nodes by combining $g(n)$ and $h(n)$, and expands to the path where $f(n)$ is

$g(n)$ = The cost to reach the node.

$h(n)$ = The cost to get from the node to goal node.

$$f(n) = g(n) + h(n)$$

~~State $g(n)$~~

which is the estimated cost of the cheapest solution

through to n . A* search expands to the path where

- $f(n)$ is the least.

We can solve the 8 puzzle problem using A* search

to reach from the initial state to goal state -

our Desired State

↳

1	2	3
8		4
7	6	5

Desired
State

↳

2		3
1	8	4
7	6	5

Initial
State

2		3
1	8	4
7	6	5

$g=0$

$h=3$

7	8	1
6		2
5	4	3

1	2	3
8		4
7	6	5

2	8	3	$g=1$
1		4	$h=3$
7	6	5	$f=4$

	2	3	$g=1$
1	8	4	$h=2$
7	6	5	$f=3$

2	3		$g=1$
1	8	4	$h=4$
7	6	5	$f=5$

1	2	3	$g=2$
	8	4	$h=1$
7	6	5	$f=3$

1	2	3
8		4
7	6	5